

Bill Payment Tracker

Project handoff • work completed and current status

Prepared October 7, 2026 • Work completed and current status

1. Project overview and work history

We built a self-hosted application for recording recurring Canadian household bills, their monthly amounts, and payments. The initial workspace was empty. The implementation includes local login, a Canadian provider directory, manual custom providers, bill accounts, payment history, spending summaries, reminders, and CSV exports.

Current status: the latest check on October 7, 2026 found no running Bill Tracker Compose containers and no HTTP response from localhost:8008. The code and deployment configuration are present, but the site was down at that check. This document does not start or change the application.

Stage	What we did	Result
Initial build • July 2, 2026	Created the project, database schema, authentication, provider/account/payment workflows, dashboard, reports, exports, and README.	Application and deployment files created from scratch.
Implementation verification	Generated and applied migrations; seeded 34 Canadian providers; exercised pages and exports; built the image; tested login/dashboard in the browser.	Nine tests passed locally and inside the production image during the earlier build.
Port troubleshooting	Diagnosed host port 8000 being occupied. A temporary acceptance run used 18000; the user then chose 8008.	Host port 8008 now maps to container port 8000.
Deployment on 8008	Updated the environment, example configuration, Compose port mapping, and README URL.	Earlier run showed healthy Postgres, running Gunicorn, and HTTP 200 from the login page inside the container.
Latest availability check • October 7, 2026	Checked Compose status and localhost:8008 outside the restricted sandbox.	No running project containers; HTTP connection failed.

Architecture decisions

We selected Django with server-rendered HTML/CSS and local JavaScript charts to keep authentication, validation, migrations, and business logic in one service. Gunicorn serves the application. PostgreSQL is the Docker database; SQLite is the fallback when DATABASE_URL is absent for local development.

The current image uses Python 3.13. Dependency pins are Django 5.2.4, Gunicorn 23.0.0, psycopg 3.2.9, and WhiteNoise 6.9.0. These are the repository's existing pins, not a claim that a dependency update audit has been completed.

2. Implemented application capabilities

Area	What is implemented
Authentication and profile	Username/password login, POST logout, protected application routes, initial admin creation from environment variables, profile editing, and password changes.
Provider directory	Searchable list of 34 seeded Canadian providers, including hydro, gas, telecom, and municipal services. Built-in and custom labels appear in the provider list.
Custom providers	Create and edit name, category, optional website/phone/region/notes, and active status. Custom providers are saved immediately and available in the account selector. Duplicate custom names are checked per owner.
Bill accounts	Create, list, edit, search, and delete accounts with provider, account number, email, service address, city, province, postal code, country, billing frequency, typical amount, due day, auto-pay, active status, and notes.
Monthly payments	One record per account and billing month/year, including amount due/paid, payment and due dates, status, method, reference, notes, and optional attachments.
Dashboard	Current billing-month and year paid totals, current-month outstanding balance, active account count, 12-month spending line chart, paid/open donut, category/provider spending lists, and upcoming/overdue bill lists.
Reports	Filtered paid total, average recorded month, highest bill in the selection, current versus previous month change, upcoming balance, overdue balance, monthly line chart, category/provider spending lists.
Filters and search	Report/payment filters for month, year, provider, category, stored payment status, province, and address. Account search covers provider, account number, email, address, postal code, and category.
Reminders	Views for bills due in seven days, bills due in 30 days, and overdue bills.
Export	All account records to CSV; payment history to CSV with the report/payment filters applied.
Interface	Responsive pages for login, dashboard, accounts, providers, payments, reports, reminders, and settings; mobile navigation and form validation feedback.

Supported categories are electricity, natural gas, water, internet, mobile, home phone, TV/cable, insurance, property tax, mortgage/rent, condo fees, credit cards, loans, subscriptions, and other. Categories are fixed choices in code, rather than a separate editable Categories database table.

3. Data, security, and deployment

Database and ownership

Django's User model handles local identities. Provider records may be shared built-ins or custom records owned by a user. Each BillAccount references a provider; each BillPayment references an account; each Attachment references a payment. The initial schema is stored in `tracker/migrations/0001_initial.py`.

Application views scope accounts, payments, custom-provider edits, exports, and attachment downloads to the logged-in owner. Normal provider-management pages do not allow editing or deleting seeded providers. The Django superuser admin is a separate privileged surface and can manage registered models.

An in-use provider is protected from deletion. Deleting an account also deletes its payment history and related attachment database records. Physical uploaded-file cleanup after record deletion is not implemented.

Security work completed

- Django password hashing, session authentication, password validators, and CSRF middleware/forms.
- Input handling through model forms, ownership checks, and account/month/year uniqueness for payment records.
- POST-only delete/logout operations, with delete confirmation in the interface.
- Secrets supplied through environment variables; `.env` excluded from Git and Docker build context.
- Upload extensions restricted to PDF, PNG, JPG/JPEG, and WebP, with a 10 MB form limit and randomized disk filenames.
- Attachments downloaded through an authenticated ownership check; no public media-serving route.
- Application image runs as a non-root user. WhiteNoise serves static assets.

The app does not store external utility-site passwords. Existing HTTPS-related settings can be configured; a reverse proxy and HTTPS deployment have not been installed or verified in this work. Upload validation checks extension and size, rather than scanning file contents. No independent security audit was performed.

Docker configuration

Setting	Current configuration
App service	Django/Gunicorn, built from Dockerfile; startup script runs migrations, provider seed, admin bootstrap, and static collection.
Database service	postgres:17-alpine; app waits for the PostgreSQL health check. Database has no host-published port.
Website URL	<code>http://localhost:8008</code>
Port mapping	Host 8008 → container 8000; <code>APP_PORT</code> controls the host side.
Trusted origin	<code>CSRF_TRUSTED_ORIGINS=http://localhost:8008</code>
Allowed hosts	localhost and 127.0.0.1 in the current environment configuration.
Persistent storage	<code>postgres_data</code> for the database; uploads for <code>/app/media</code> .
Restart policy	unless-stopped for app and database.

The port fix changed the host mapping while retaining Gunicorn's internal port 8000. Both `.env` and `.env.example` now use `APP_PORT=8008`. Credentials and secret values are intentionally omitted from this document.

4. Verification evidence and remaining work

Earlier verification results

The following checks were completed during the earlier implementation and deployment work. They are historical results; the nine-test suite was not rerun when preparing this document.

- Framework system check passed with no issues, and migration drift check reported no changes.
- Initial migration applied successfully; provider seeding created 34 records.
- Pages and exports rendered with both empty and sample account/payment data.
- Production Docker image built successfully on Python 3.13/Django 5.2.
- Nine tests passed locally and in the Docker image: anonymous-route protection; page/export rendering; seeded and other-user provider edit denial; other-user payment denial; custom-provider selector availability; custom-provider account/payment creation; rejection of another user's provider; filtered CSV generation; and attachment ownership enforcement.
- Compose startup exercised PostgreSQL health, migrations, seed, admin creation, static collection, and Gunicorn.
- Browser verification confirmed login and the dashboard with no reported console errors during the temporary acceptance run.
- The subsequent 8008 deployment showed the correct mapping and an internal login HTTP 200.

Limits and follow-up items identified from the current code

1. **Status consistency:** overdue display can be calculated from the due date, but the status filter queries the stored status. Paid/unpaid totals also rely on stored status. Amounts, dates, and status are not fully cross-validated.
2. **Reporting scope:** category/provider results are numeric lists, not the requested category/provider bar or pie charts. There is no dedicated highest-provider yearly KPI card. The highest-bill card follows selected filters, not automatically the current month.
3. **CSV scope:** report export returns filtered payment rows. A separate aggregated monthly/yearly report CSV has not been implemented. The existing CSV test verifies expected output but does not prove exclusion of every nonmatching row.
4. **Spending definitions:** totals use the billing month/year, not the actual payment-date month. Average monthly spend includes only periods with records. Month-over-month compares the current and prior calendar billing months regardless of report filters.
5. **Manual workflow:** billing frequency and due day are stored, but recurring bill entries are not generated automatically. Each payment record stores a cumulative amount paid, rather than a separate ledger of multiple payment transactions.
6. **Other boundaries:** CSV import, email notifications, audit logging, bank integrations, and utility-site scraping are absent. No container-restart data-retention test was recorded; named volumes are configured for persistence. Dependency updates and stronger validation remain follow-up work.

These observations refine the earlier broad v1-completion summary. The main workflows exist, while the items above remain to be addressed or explicitly accepted.

5. Operating the project and continuing work

Project directory: /Users/chandu/Chandra-projects/bill_tracker

Start and check availability

Run in the project directory, using the existing .env:

```
docker compose up -d
docker compose ps
docker compose logs --tail=80 app
curl -I http://localhost:8008/login/
```

After source changes, rebuild with `docker compose up -d --build`. Open `http://localhost:8008` and sign in using the admin credentials established during initial setup. The bootstrap command does not overwrite an existing user's password; changing `ADMIN_PASSWORD` later is not a password reset.

Routine validation

```
docker compose config --quiet
docker compose exec app python manage.py check
docker compose exec app python manage.py makemigrations --check --dry-run
docker compose exec app python manage.py test tracker
```

These commands assume the app and database are running. If the app exits during startup, inspect its logs first.

Stop and back up

```
docker compose exec -T db pg_dump -U bill_tracker bill_tracker > bill_tracker_backup.sql
docker compose down
```

Back up before stopping the stack, while the database is running. Substitute your configured database/user names if different. Uploads need their own volume backup. Normal container recreation preserves the named volumes; `docker compose down -v` removes them and their data.

Key files

File or directory	Purpose
README.md	Setup, usage, verification, backups, and operational guidance.
docker-compose.yml, Dockerfile, entrypoint.sh	Services, image build, persistence, and startup sequence.
.env.example	Environment template; .env holds the local configuration.
bill_tracker/settings.py	Database, session, CSRF, static/upload, and deployment settings.
tracker/models.py, forms.py, views.py, urls.py	Data model, input handling, application behavior, and routes.
tracker/management/commands/	Provider seeding and initial admin bootstrap.
tracker/migrations/, tracker/tests.py	Schema migration and nine regression tests.
templates/, static/	Application pages, responsive styling, and chart renderer.

Suggested next sequence: start and verify the existing deployment; review the reporting and validation gaps above; confirm credentials are configured for actual use; then verify persistence across a restart and establish database/upload backups. The original README remains available for detailed setup instructions; this document records the work and the latest known status.